

Name: Javaughn Stevens

CMSC 204 Project 6 Graphs Pseudocode (my high-level design)

// Pseudocode for Town.java

Declare a public class Town that implements Comparable<Town>

Declare a private String variable townName initialized to empty string

Declare a public constructor Town with no parameters

Set this.townName equal to null

// end of constructor

Declare a public constructor Town with String parameter argument townName

Set this.townName equal to townName

// end of constructor

Declare a public void method setTownName with String parameter argument townName

Set this.townName equal to townName

// end of setTownName method

Declare a public String method getName with no parameters

Return townName

// end of getName method

Declare a public String method toString with no parameters

Return townName

// end of toString method

Declare a public integer method compareTo with Town parameter
argument otherTown

If otherTown is null
Return 1

If this.townName is null AND otherTown.townName is null
Return 0

Else if this.townName is null
Return -1

Else if otherTown.townName is null
Return 1

Else
If this.townName.compareTo(otherTown.townName) is less than 0
Return -1
Else if this.townName.compareTo(otherTown.townName) is
greater than 0
Return 1
Else
Return 0
// end of compareTo method

Declare a public boolean method equals with Object parameter
argument otherTown

If this is equal to otherTown
Return true

If otherTown is not an instance of Town

Return false

Declare Town variable newTown and set it equal to cast of otherTown

If this.townName.equalsIgnoreCase(newTown.townName) is true

Return true

Else

Return false

// end of equals method

// hashCode method

Declare a public integer method hashCode with no parameters

Declare integer variable hashCode equal to 17

If townName is null

Set hashCode equal to 31 * hashCode

Else

Set hashCode equal to 31 * hashCode + townName.hashCode()

Return hashCode

// end of hashCode method

// end of class Town and Town.java pseudocode

// Pseudocode for Road.java

// pseudocode for Road.java

Declare a public class Road that implements Comparable<Road>

Declare a private integer variable distance initialized to 0

Declare a private Town variable start initialized to new Town

Declare a private Town variable destination initialized to new
Town

Declare a private String variable roadName initialized to empty
string

Declare a public constructor Road with no parameters

Set start equal to null

Set distance equal to 0

Set destination equal to null

Set roadName equal to null

// end of constructor

Declare a public constructor Road with parameters Town start,
Town destination, integer distance, String roadName

Set this.start equal to start

Set this.destination equal to destination

Set this.distance equal to distance

Set this.roadName equal to roadName

// end of constructor

Declare a public Town method getStart with no parameters

Return start

// end of getStart method

Declare a public void method setStart with Town parameter start

Set this.start equal to start

// end of setStart method

Declare a public Town method getDestination with no parameters

Return destination

// end of getDestination method

Declare a public void method setDestination with Town parameter destination

Set this.destination equal to destination

// end of setDestination method

Declare a public integer method getDistance with no parameters

Return distance

// end of getDistance method

Declare a public void method setDistance with integer parameter distance

Set this.distance equal to distance

// end of setDistance method

Declare a public String method getName with no parameters

Return roadName

// end of getName method

Declare a public void method setName with String parameter roadName

Set this.roadName equal to roadName

// end of setName method

Declare a public integer method compareTo with Road parameter otherRoad

If otherRoad is null

Return 1

If this.distance is equal to 0 AND otherRoad.distance is equal to 0

Return 0

Else if this.distance is equal to 0

Return -1

Else if otherRoad.distance is equal to 0

Return 1

Else

If this.distance is less than otherRoad.distance

Return -1

Else if this.distance is greater than otherRoad.distance

Return 1

Else

Return 0

// end of compareTo method

Declare a public boolean method equals with Object parameter
otherRoad

If this is equal to otherRoad

Return true

If otherRoad is not an instance of Road

Return false

Declare Road variable newRoad and set it equal to cast of
otherRoad

If this.start is null OR this.destination is null OR newRoad.start is
null OR newRoad.destination is null

Return false

Declare boolean condition1 equal to false
Declare boolean condition2 equal to false

If this.start.equals(newRoad.start) is true
Set condition1 equal to true

If this.destination.equals(newRoad.destination) is true
Set condition2 equal to true

If condition1 is true AND condition2 is true
Return true

Set condition1 equal to false
Set condition2 equal to false

If this.start.equals(newRoad.destination) is true
Set condition1 equal to true

If this.destination.equals(newRoad.start) is true
Set condition2 equal to true

If condition1 is true AND condition2 is true
Return true

Else

Return false

// end of equals method

Declare a public integer method hashCode with no parameters

Declare integer variable hash1

Declare integer variable hash2

```
If start is null
Set hash1 equal to 0
Else
Set hash1 equal to start.hashCode()
```

```
If destination is null
Set hash2 equal to 0
Else
Set hash2 equal to destination.hashCode()
```

```
Declare integer variable totalHash equal to 17
Set totalHash equal to 31 * totalHash + (hash1 + hash2)
```

```
Return totalHash
// end of hashCode method
```

```
Declare a public String method toString with no parameters
Return roadName + " (" + distance + " miles) from " + start + "
to " + destination
// end of toString method
```

```
// end of class Road and Road.java pseudocode
```

```
// Pseudocode for Graph.java
```

```
Declare a public class Graph that implements
GraphInterface<Town, Road>
```

```
Declare a private Set<Town> variable townVertices
Declare a private Set<Road> variable roadEdges
Declare a private Map<Town, Integer> variable distances
Declare a private Map<Town, Town> variable previousVertices
```


Declare a public constructor Graph with no parameters
Set townVertices equal to new empty HashSet
Set roadEdges equal to new empty HashSet
// end of constructor

Declare a public Road method getEdge with parameters Town
startVertex, Town destinationVertex

For each Road edgeRoad in roadEdges
If edgeRoad.start equals startVertex AND edgeRoad.destination
equals destinationVertex
OR edgeRoad.start equals destinationVertex AND
edgeRoad.destination equals startVertex
Return edgeRoad

Return null
// end of getEdge method

Declare a public Road method addEdge with parameters Town
startVertex, Town destinationVertex, integer weight, String
description

If townVertices does not contain startVertex OR townVertices
does not contain destinationVertex
Throw IllegalArgumentException

Declare Road newRoad equal to new Road(startVertex,
destinationVertex, weight, description)
Add newRoad to roadEdges
Return newRoad
// end of addEdge method

Declare a public boolean method addVertex with parameter Town
newVertex

If newVertex is null
Throw NullPointerException

If townVertices.add(newVertex) is true
Return true
Else
Return false
// end of addVertex method

Declare a public boolean method containsEdge with parameters
Town startVertex, Town destinationVertex
If getEdge(startVertex, destinationVertex) is not null
Return true
Else
Return false
// end of containsEdge method

Declare a public boolean method containsVertex with parameter
Town vertex
If vertex is not null AND townVertices contains vertex
Return true
Else
Return false
// end of containsVertex method

Declare a public Set<Road> method edgeSet with no parameters
Return new HashSet copy of roadEdges
// end of edgeSet method

// edgesOf method

```
Declare a public Set<Road> method edgesOf with parameter
Town vertex
If townVertices does not contain vertex
Throw IllegalArgumentException
Declare Set<Road> connectedRoads equal to new empty HashSet
For each Road edgeRoad in roadEdges
If edgeRoad.start equals vertex OR edgeRoad.destination equals
vertex
Add edgeRoad to connectedRoads

Return connectedRoads
// end of edgesOf method
```

```
Declare a public boolean method removeVertex with parameter
Town vertex
If townVertices does not contain vertex
Return false
Remove from roadEdges all edges where edge.start equals vertex
OR edge.destination equals vertex
If townVertices.remove(vertex) is true
Return true
Else
Return false
// end of removeVertex method
```

```
Declare a public Set<Town> method vertexSet with no
parameters
Return new HashSet copy of townVertices
// end of vertexSet method
```

```
Declare a public ArrayList<String> method shortestPath with
parameters Town startVertex, Town destinationVertex
```

Call method dijkstraShortestPath(startVertex)

Declare ArrayList<String> path equal to new empty ArrayList

Declare Town currentTown equal to destinationVertex

If previousVertices.get(currentTown) is null AND currentTown is not equal to startVertex

Return path

While currentTown is not equal to startVertex

Declare Town previousTown equal to previousVertices.get(currentTown)

Declare Road newRoad equal to getEdge(previousTown, currentTown)

Append to index 0: previousTown + " via " + newRoad.name + " to " + currentTown + " " + newRoad.distance + " mi"

Set currentTown equal to previousTown

Return path

// end of shortestPath method

Declare a public void method dijkstraShortestPath with parameter Town startVertex

Set distances equal to new empty HashMap

Set previousVertices equal to new empty HashMap

For each Town pathTown in townVertices

Set distances[pathTown] equal to infinity

Set previousVertices[pathTown] equal to null

Set distances[startVertex] equal to 0

Declare a PriorityQueue<Town> pathQueue ordered by distances
Add startVertex to pathQueue

While pathQueue is not empty

Declare Town currentTown equal to pathQueue.poll()

For each Road road in edgesOf(currentTown)

If road.start equals currentTown

Set neighborTown equal to road.destination

Else

Set neighborTown equal to road.start

Declare integer newDistance equal to distances[currentTown] +
road.distance

If newDistance is less than distances[neighborTown]

Set distances[neighborTown] equal to newDistance

Set previousVertices[neighborTown] equal to currentTown

Remove neighborTown from pathQueue

Add neighborTown to pathQueue

// end of dijkstraShortestPath method

Declare a public Road method removeEdge with parameters Town
startVertex, Town destinationVertex, integer distance, String
description

Declare Road existingRoad equal to getEdge(startVertex,
destinationVertex)

If existingRoad is not null AND (distance is less than 0 OR
existingRoad.distance equals distance)
AND (description is null OR existingRoad.name equals
description)

Remove existingRoad from roadEdges
Return existingRoad

Return null
// end of removeEdge method

Declare a public Road method peekEdge with parameters Town
startVertex, Town destinationVertex, integer distance, String
description

Declare Road existingRoad equal to getEdge(startVertex,
destinationVertex)

If existingRoad is not null AND (distance is less than 0 OR
existingRoad.distance equals distance)
AND (description is null OR existingRoad.name equals
description)

Return existingRoad

Return null
// end of peekEdge method

// end of class Graph and Graph.java pseudocode

// Pseudocode for TownGraphManager.java

Declare a public class TownGraphManager that implements
TownGraphManagerInterface

Declare a private Graph variable managerGraph

Declare a public constructor TownGraphManager with no
parameters

Set managerGraph equal to new Graph

// end of constructor

Declare a public boolean method addRoad with parameters String
town1, String town2, integer distance, String roadName

If town1 is null OR town2 is null OR roadName is null

Return false

Declare Town t1 equal to new Town(town1)

Declare Town t2 equal to new Town(town2)

If managerGraph does not contain vertex t1

Call method managerGraph.addVertex(t1)

If managerGraph does not contain vertex t2

Call method managerGraph.addVertex(t2)

If managerGraph.addEdge(t1, t2, distance, roadName) is not null

Return true

Else

Return false

// end of addRoad method

Declare a public String method getRoad with parameters String town1, String town2

Declare Road existingRoad equal to managerGraph.getEdge(new Town(town1), new Town(town2))

If existingRoad is not null
Return existingRoad.name
Else
Return null
// end of getRoad method

Declare a public boolean method addTown with parameter String town
If town is null
Return false
If managerGraph.addVertex(new Town(town)) is true
Return true
Else
Return false
// end of addTown method

Declare a public Town method getTown with parameter String town
Declare Town temporary equal to new Town(town)
For each Town existingTown in managerGraph.vertexSet
If existingTown equals temporary
Return existingTown

Return null
// end of getTown method

Declare a public boolean method containsTown with parameter String town

If managerGraph.containsVertex(new Town(town)) is true

Return true

Else

Return false

// end of containsTown method

Declare a public boolean method containsRoadConnection with parameters String town1, String town2

If managerGraph.containsEdge(new Town(town1), new Town(town2)) is true

Return true

Else

Return false

// end of containsRoadConnection method

Declare a public ArrayList<String> method allRoads with no parameters

Declare ArrayList<String> roadList equal to new empty ArrayList

For each Road existingRoad in managerGraph.edgeSet

Add existingRoad.name to roadList

Sort roadList

Return roadList

// end of allRoads method

Declare a public boolean method deleteRoadConnection with parameters String town1, String town2, String road

Declare Road existingRoad equal to managerGraph.getEdge(new Town(town1), new Town(town2))

If existingRoad is not null AND existingRoad.name equals road
Call managerGraph.removeEdge(new Town(town1), new Town(town2), existingRoad.distance, road)
Return true

Return false
// end of deleteRoadConnection method

Declare a public boolean method deleteTown with parameter String town
If managerGraph.removeVertex(new Town(town)) is true
Return true
Else
Return false
// end of deleteTown method

Declare a public ArrayList<String> method allTowns with no parameters
Declare ArrayList<String> townList equal to new empty ArrayList
For each Town existingTown in managerGraph.vertexSet
Add existingTown.name to townList

Sort townList

Return townList
// end of allTowns method

Declare a public ArrayList<String> method getPath with parameters String town1, String town2

Declare Town start equal to new Town(town1)
Declare Town destination equal to new Town(town2)

If managerGraph does not contain start OR managerGraph does not contain destination

Return new empty ArrayList

Else

Return managerGraph.shortestPath(start, destination)

// end of getPath method

Declare a public void method populateTownGraph with parameter File selectedFile

Throws FileNotFoundException, IOException

Declare Scanner fileScanner equal to new Scanner(selectedFile)

Try

While fileScanner has next line

Declare String line equal to next line trimmed

If line is empty

Continue loop

Declare String array townParts equal to split line by ";"

If length of townParts is not equal to 3

Throw IOException

Declare String array roadInfo equal to split townParts[0] by ","

If length of roadInfo is not equal to 2

Throw IOException

Declare String roadName equal to trimmed roadInfo[0]

Declare integer distance

Try

Set distance equal to parse integer of roadInfo[1]

Catch NumberFormatException

Throw IOException

Declare String town1 equal to trimmed townParts[1]

Declare String town2 equal to trimmed townParts[2]

Call addTown(town1)

Call addTown(town2)

Call addRoad(town1, town2, distance, roadName)

Catch FileNotFoundException

Print stack trace

Finally

Close fileScanner

// end of populateTownGraph method

// end of class TownGraphManager and TownGraphManager.java
pseudocode